

Security & Code Quality Audit Report

bateumz-cb2c44d1 | React + TypeScript + Supabase

August 12, 2026

Generated by Z.ai Code Auditor

Table of Contents

1. Executive Summary
2. Audit Scope & Methodology
3. Critical Findings (11 issues)
4. High Severity Findings (19 issues)
5. Medium Severity Findings (21 issues)
6. Low Severity Findings (19 issues)
7. Issue Distribution Summary
8. Priority Remediation Roadmap

1. Executive Summary

A comprehensive security and code quality audit was performed on the **bateumz-cb2c44d1** project, a React + TypeScript single-page application built on Supabase for authentication, database, and real-time features. The application is a raffle, contest, and live-gaming platform with wallet functionality, payment processing (Stripe, PayPal), esports tournaments, and a multi-region theming system. The codebase comprises approximately 250+ source files including 70+ live game components, 20+ admin dashboard pages, 15+ contexts/hooks, and 40+ library modules.

The audit identified **70 distinct issues** across security vulnerabilities, type safety gaps, logic bugs, error handling deficiencies, and code quality concerns. Of these, **11 are rated CRITICAL**, representing exploitable vulnerabilities that could lead to financial loss, authentication bypass, or arbitrary code execution. An additional **19 issues are rated HIGH**, including race conditions in financial operations, missing authorization checks, and cross-site scripting vectors. The remaining issues are MEDIUM (21) and LOW (19) severity, covering input validation gaps, pervasive use of unsafe type casts, performance concerns, and minor logic errors.

The most urgent findings involve: (1) SQL injection via string interpolation in esports placement submission, (2) multiple race conditions in financial operations (wallet withdrawals, voucher redemption, battle wagers) that could enable double-spending, (3) stored XSS via unsanitized CSS injection from the database, (4) client-side role escalation during user registration, and (5) open redirect vulnerabilities in OAuth flows. These issues require immediate remediation before the platform processes real financial transactions at scale.

Severity	Count	Key Categories
CRITICAL	11	SQL Injection, XSS, Race Conditions, Auth Bypass, Open Redirect
HIGH	19	XSS, Broken Features, Missing Auth, Insecure RNG, Type Safety
MEDIUM	21	Input Validation, Logic Bugs, Privacy, Memory Leaks, Type Safety
LOW	19	Error Handling, Dead Code, Code Quality, Resource Leaks

2. Audit Scope & Methodology

The audit covered all source files within the **src/** directory of the project, organized into the following modules. Each file was manually reviewed for security vulnerabilities, type safety issues, logic bugs, error handling patterns, and code quality concerns. The audit was performed statically by reading source code; no dynamic testing or penetration testing was conducted. Findings are categorized by severity using a standard four-tier system.

Module	Files Audited	Focus Areas
src/integrations/supabase/	client.ts, types.ts	Client config, type definitions
src/contexts/	7 files	Auth, language, theme, currency, regional
src/hooks/	8 files	Regional config, push notifications, tracking
src/lib/	44 files	Wallet, stripe, audit, esports, live platform
src/components/	150+ files	Payments, wallet, live games, auth, uploads
src/pages/	90+ files	Auth, admin, dashboard, user pages
src/layouts/	2 files	Admin, dashboard layout wrappers
src/App.tsx, main.tsx	2 files	Routing, provider nesting, error boundaries

3. Critical Findings

Critical findings represent exploitable vulnerabilities that could result in financial loss, authentication bypass, arbitrary code execution, or data theft. These issues require **immediate remediation** before any production deployment.

CRITICAL

#1 SQL Injection via `sb.raw()` with String Interpolation

Security

`src/lib/esports.ts` (lines 1020-1028)

The `submitPlacements()` function interpolates `pts`, `p.kills`, and `p.deaths` directly into raw SQL strings via `sb.raw()`. These values originate from client-supplied function parameters, enabling a malicious client to inject arbitrary SQL commands. For example, a `kills` value of `"1); DROP TABLE user_battles;--"` would be interpolated directly into the SQL statement. This is a classic SQL injection vulnerability in a financial feature (esports betting).

Fix: Use parameterized RPC calls instead of raw SQL. At minimum, validate that all interpolated values are integers (`Number.isInteger()`). Ideally, move this logic into a Postgres function called via `sb.rpc()`.

CRITICAL

#2 Race Condition in Voucher Redemption (Double-Spend)

Logic Bug / Financial

`src/lib/vouchers.ts` (lines 107-148)

The `applyVoucher()` function performs three non-atomic operations: (1) validates the voucher, (2) inserts a redemption record, and (3) increments `current_uses`. Steps 2 and 3 are separate Supabase requests. If two concurrent requests hit simultaneously, both pass validation with the same `current_uses` value, both insert redemptions, and both increment the counter. This bypasses the `max_uses` limit, allowing unlimited use of a voucher intended to be single-use or limited. The code includes a fallback path (when the RPC does not exist) that is inherently non-atomic.

Fix: Remove the non-atomic fallback entirely. Use **ONLY** the `increment_voucher_uses` RPC, which should be an atomic `UPDATE-RETURNING` operation. If the RPC does not exist, create it as a server-side function with proper transaction handling.

CRITICAL

#3 Race Condition in Wallet Withdrawal (Double-Spend)

Logic Bug / Financial

`src/lib/wallet.ts` (lines 171-201)

The `createWithdrawalRequest()` function performs a balance check via `getBalance()` (a read operation), then calls `processTransaction()` to debit the account, then inserts a withdrawal request record. Between the balance check and the debit, another tab or concurrent request could initiate a separate withdrawal. Both requests would pass the balance check using a stale read, resulting in a negative balance. This is a time-of-check-to-time-of-use (TOCTOU) race condition on a direct financial operation.

Fix: The `wallet_process_transaction` RPC must enforce the balance check atomically server-side using `SELECT ... FOR UPDATE` or equivalent. Remove or downgrade the client-side

balance check to a UX-only pre-check. Ensure the RPC is the single source of truth for balance enforcement.

CRITICAL

#4 Race Condition in Battle Wager (Double-Spend)

Logic Bug / Financial

`src/lib/battles.ts` (lines 52-68)

The `createBattle()` function reads the user balance client-side via `getBalance()`, then calls the `create_battle` RPC. The same TOCTOU issue as wallet withdrawal applies: two concurrent battle creation requests could both pass the balance check before either wager is deducted, allowing a user to wager more than their actual balance. Additionally, there is no minimum wager validation against the game defined `minBet`.

Fix: The `create_battle` RPC must enforce the balance check atomically. Validate `wagerAmount >= minBet` on the client as UX guidance, and enforce it in the server-side RPC.

CRITICAL

#5 Stored XSS via Arbitrary CSS Injection from Database

Security

`src/contexts/RegionalThemeContext.tsx` (lines 61-66)

The `applyCssVars()` function injects `theme.custom_css` from the database directly into a DOM style element via `style.textContent = theme.custom_css`. A malicious admin who controls the regions table can inject any CSS, including `background-image: url()` for data exfiltration, `@import url()` for external resource loading, or `CSS expression()` for code execution in legacy browsers. The comment on line 64 claims to "scope under `:root`" but the raw CSS is injected verbatim with no sanitization whatsoever.

Fix: Sanitize all CSS content before injection using a library like `DOMPurify` configured for CSS, or a dedicated CSS parser (e.g., `css-tree`). At minimum, strip `@import`, `url()`, `expression()`, and behavior patterns from the CSS string.

CRITICAL

#6 Stored XSS via CSS Injection in useRegionalConfig

Security

`src/hooks/useRegionalConfig.tsx` (lines 225-231)

Identical XSS vector to issue #5. The `applyRegionalBranding()` function sets `styleElement.innerHTML = branding.custom_css` where `branding.custom_css` comes from the `regional_branding` table. No sanitization is performed before injection into the DOM. Any user with database access to this table (or who can manipulate the API response) can execute arbitrary CSS-based attacks.

Fix: Apply the same CSS sanitization as issue #5. Both injection points must be fixed simultaneously since they serve the same purpose in different code paths.

CRITICAL

#7 Client-Side Role Escalation in Signup Metadata

Security

`src/contexts/AuthContext.tsx` (lines 190-203)

The `signUp()` function accepts a `meta.role` parameter from the caller and writes it directly into `user_metadata.data.role`. While the Register page only offers "user" or "business", the

Supabase auth endpoint accepts arbitrary metadata. If any server-side trigger or edge function reads `user_metadata.role` to populate the `user_roles` table, a user could pass `role: "admin"` or `role: "superadmin"` during registration to self-assign elevated privileges.

Fix: Remove the `role` parameter from the client-side `signUp` metadata entirely, or hardcode it to `"user"` on the client. Role assignment should only occur server-side via admin action or a database trigger that ignores client-supplied metadata.

CRITICAL #8 Open Redirect via OAuth next Parameter Security

`src/pages/Login.tsx` (lines 34-40, 125)

The `nextPath` is parsed from the `?next=` query parameter with only a basic guard against protocol-relative URLs (starts with `"/"`). This validation is insufficient: paths like `"/@evil.com"` or backslash-encoded variants could bypass the check. The value is directly concatenated into the OAuth `redirectTo` URL. An attacker could craft a phishing link that redirects users to a malicious site after authentication, potentially capturing OAuth tokens.

Fix: Whitelist allowed redirect paths (e.g., `/profile`, `/dashboard`, `/admin`) instead of allowing any path starting with `"/"`. Use `new URL(redirectTo, origin)` and verify the origin matches `window.location.origin`.

CRITICAL #9 Unvalidated Redirect URL in OAuth Consent Flow Security

`src/pages/OAuthConsent.tsx` (lines 39-43, 61, 67)

After OAuth authorization approval, the page redirects to `data?.redirect_url` without validating it against a registered list of allowed redirect URIs. Additionally, lines 39-43 auto-redirect if `redirect_url` exists and `data?.client` is falsy, bypassing user consent entirely. A malicious OAuth client could specify an attacker-controlled `redirect_url` to intercept authorization codes.

Fix: Validate the `redirect_url` against the registered OAuth client allowed redirect URIs before performing the redirect. Never auto-redirect without explicit user consent.

CRITICAL #10 No Server-Side Authorization on Admin Routes Security

`src/layouts/AdminLayout.tsx` / `src/components/ProtectedRoute.tsx` (lines all / 29-30)

The `AdminLayout` component never independently verifies the user role server-side. `ProtectedRoute` checks the role from `AuthContext`, which is populated by a client-side Supabase query to the `user_roles` table. If RLS policies on `user_roles` are misconfigured or absent, an attacker could insert a row granting themselves admin access. The admin sidebar renders links to all admin pages including user management, revenue data, and system settings.

Fix: Add a server-side verification step (e.g., a `security_definer` RPC function) within the admin route protection. Ensure `user_roles` has RLS enabled and `INSERT` is restricted to server-side functions only.

CRITICAL

#11 XSS via document.write() with Unsanitized Player Names

Security

`src/components/livegames/LiveControlPanel.tsx (lines 1378, 1387-1389)`

The `printReport()` function constructs raw HTML via template literals and writes it into a new browser window via `w.document.write(html)`. While some table row data sanitizes names with `.replace(/[<>&]/g, "")`, the podium player names on lines 1387-1389 and the `liveCode` on line 1378 are interpolated without any sanitization. A malicious display name like `<img src=x onerror=alert(document.cookie)>` would execute in the same origin context.

Fix: Apply consistent HTML-escaping (or `DOMPurify`) to ALL user-derived values before interpolation into the HTML template, including podium names, `liveCode`, and any other dynamic content.

4. High Severity Findings

High severity findings include exploitable vulnerabilities and significant logic bugs that could lead to data corruption, financial inconsistencies, or degraded security. These should be addressed immediately after critical issues.

HIGH

#12 SVG File Upload Enables Stored XSS

Security

`src/lib/upload-utils.ts, src/components/ImageUpload.tsx (lines 7, 13, 121)`

SVG files are explicitly allowed in the upload configuration. SVG files can contain embedded `<script>` tags, onload handlers, and foreignObject elements that execute JavaScript. If uploaded SVGs are served from Supabase Storage with Content-Type: image/svg+xml and a user navigates to the URL, arbitrary JavaScript executes in the origin context.

Fix: Either remove .svg from allowed extensions, parse uploaded SVGs server-side to strip dangerous elements, or serve SVGs from a separate sandboxed domain with Content-Disposition: attachment.

HIGH

#13 Arbitrary File Upload with No Validation

Security

`src/components/FileUpload.tsx (lines 23, 29-44)`

FileUpload has zero file validation. The accept prop defaults to `"*/*"`. No MIME type check, no file extension check, no file size check. A user can upload HTML files with embedded JavaScript or any malicious content directly to Supabase Storage. If the bucket is publicly accessible, this enables stored XSS. The path uses a flat namespace with no user scoping.

Fix: Add file type validation (extension + MIME type whitelist), file size limits, scope uploads to user folders, and block dangerous file types (HTML, HTA, SVG, JS).

HIGH

#14 Client-Side Stripe Payment Amount Recording

Security

`src/components/payments/StripeCheckout.tsx, src/lib/stripe.ts (lines 87-94, 72-90)`

After Stripe payment succeeds, `recordStripePayment` writes the amount and currency from React component props to the database. These are client-controllable values. While Stripe enforces the actual charge server-side, the database record could be manipulated to show a different amount than what was actually charged, creating fraudulent audit trails.

Fix: Fetch the actual amount from Stripe API using the `paymentIntentId` on the server side. Have the edge function handle recording after payment confirmation rather than trusting client-side values.

HIGH

#15 XSS via `textarea.innerHTML` in openTriviaDB

Security

`src/lib/openTriviaDB.ts (lines 43-47)`

The `decodeHTMLEntities()` function sets `textarea.innerHTML = text` to decode HTML entities from the Open Trivia DB API. If the API were compromised or the response tampered with via MITM, this could execute arbitrary JavaScript in the user browser.

Fix: Use DOMParser or a dedicated entity decoder library instead of innerHTML for entity decoding.

HIGH

#16 XSS via document.write() in liveHistory PDF Functions

Security

src/lib/liveHistory.ts (lines 107-114, 176-185)

printGameAggregatePDF() and printSessionsPDF() build HTML strings with template literals embedding user-provided data (game names, player names, winner metadata). These are injected via w.document.write(html) without sanitization.

Fix: Sanitize all user data before interpolating into HTML using DOMPurify or HTML entity escaping.

HIGH

#17 Bingo Draw Uses Client-Side Math.random()

Security / Logic

src/lib/livePlatform.ts (lines 479-493)

drawBingoNumber() reads game state, picks a random number client-side with Math.random(), then writes it back. This is not cryptographically secure and is a race condition: two admins could draw simultaneously and both pick the same number.

Fix: Move draw logic to a server-side RPC using pgcrypto.random() for atomic number generation.

HIGH

#18 Duel Voting Has No Duplicate Protection

Logic Bug

src/lib/livePlatform.ts (lines 706-716)

voteDuel() increments challenger_votes or challenged_votes by 1 with no check for existing votes. Any user can vote unlimited times, including the participants themselves, by calling the function repeatedly.

Fix: Add a duel_votes table with unique constraint on (duel_id, user_id). Insert a vote record first, then increment via RPC.

HIGH

#19 updateBattleStatus Has No Authorization

Security

src/lib/battles.ts (lines 147-149)

updateBattleStatus() directly updates the user_battles table with an arbitrary status string and no authorization check. Any user can change any battle status to anything, including reverting a completed battle to open.

Fix: Restrict the status parameter to the valid union type. Move authorization logic to an RPC that verifies the caller is the battle creator or an admin.

HIGH

#20 Non-Atomic Tournament Points with Silent Error Swallowing

Logic Bug

src/lib/tournaments.ts (lines 98-139)

`addTournamentPoints()` has a fallback that manually reads and writes standings when the RPC fails. The fallback is not atomic, swallows errors silently, and if the insert fails the points are lost entirely, silently corrupting tournament standings.

Fix: Remove the fallback. If the RPC fails, throw the error. The RPC should use `INSERT ... ON CONFLICT DO UPDATE` for atomicity.

HIGH

#21 Clip Likes Always Set to 0 (Broken Feature)

Logic Bug

`src/lib/livePlatform.ts` (lines 578-589)

`toggleClipLike()` unconditionally sets `likes_count`: 0 in both the like and unlike branches. The comment says "decrement via RPC ideally" but the RPC was never implemented, so clip like counts are always reset to zero on every toggle.

Fix: Implement an RPC that atomically increments/decrements the counter. Use `sb.raw()` as temporary workaround.

HIGH

#22 Referral Code Replay / Point Farming

Security

`src/contexts/AuthContext.tsx` (lines 122-147)

The referral code is read from `localStorage` and processed on profile fetch. The `points_awarded`: 50 is hardcoded client-side. An attacker can set `localStorage("sortex_ref")` to any valid code and re-register with different emails to farm referral points.

Fix: Move referral processing entirely server-side. The client should only signal that a referral occurred, never control point amounts.

HIGH

#23 Weak Password Policy on Reset

Security

`src/pages/ResetPassword.tsx` (lines 163-165)

The password reset page only validates `password.length < 6` with no complexity requirements. Registration requires 8+ characters with letters and numbers, but reset allows a trivially weak 6-character password, undermining overall account security.

Fix: Enforce the same password policy as registration: minimum 8 characters with both letters and numbers.

HIGH

#24 No Rate Limiting or CAPTCHA on Auth Pages

Security

`src/pages/Login.tsx`, `Register.tsx`, `ForgotPassword.tsx` (lines all)

None of the authentication pages implement client-side rate limiting, lockout after failed attempts, or CAPTCHA. An attacker can brute-force credentials, spam password reset emails to any address (causing inbox DoS), or automate account creation.

Fix: Implement client-side attempt counting with exponential backoff. Integrate a CAPTCHA on login and registration forms. Track failed attempts and lock the form after 5+ failures.

HIGH

#25 Tamperable Signup Data in localStorage

Security

`src/pages/Register.tsx + AuthContext.tsx (lines 139-148, 98-118)`

Extra profile data (phone, province, city, company_name) is stored in localStorage after signup and flushed to the database on next session load. An attacker can modify these values in localStorage before they are persisted, potentially setting another user phone number or impersonating a business.

Fix: Pass extra data through the auth flow or a server-side session rather than localStorage. If localStorage must be used, validate values server-side before persisting.

HIGH

#26 ErrorBoundary Leaks Error Details to Users

Security

`src/components/ErrorBoundary.tsx (lines 49-52)`

The raw error.message is rendered in a `<pre>` tag visible to end users. Error messages can contain sensitive information including file paths, API endpoints, internal function names, and database schema details, aiding attackers in reconnaissance.

Fix: In production, display a generic error message. Only show detailed errors in development mode. Log full errors to an error-tracking service.

HIGH

#27 No Authorization Check on Translation Mutations

Security

`src/hooks/useRegionalTranslations.ts (lines 71-100, 105-136)`

`updateTranslation` and `bulkUpdateTranslations` perform upsert operations on `regional_translations` with no authorization check. Any signed-in user can modify translations for any region, despite the comment saying "only for regional CEO".

Fix: Add explicit role verification before performing writes. Verify the user owns the region being modified.

HIGH

#28 Empty catch {} Blocks Swallow Errors in Auth Flow

Error Handling

`src/contexts/AuthContext.tsx (lines 118, 146)`

Two catch blocks are completely empty in the referral processing and signup extra data parsing. If these fail (malformed JSON, network error, DB constraint violation), the error is silently discarded. For referral processing, this involves financial data.

Fix: Add `console.error()` with context at minimum. For referral processing, show a non-blocking toast notification on failure.

HIGH

#29 PayPal onApprove Has No Double-Submit Protection

Security

`src/components/payments/PayPalCheckout.tsx (lines 33-43)`

The `onApprove` handler does not set any loading or disabled state. If the PayPal callback fires multiple times, multiple `paypal-capture-order` edge function calls could be made for the same `orderId`, potentially resulting in duplicate captures.

Fix: Add a local capturing state flag set to true at the start of onApprove. Check it before processing. Disable PayPal buttons while capturing.

HIGH

#30 Deposit Receipt Upload Has No Validation

Security

`src/components/wallet/DepositModal.tsx` (lines 64-71)

The receipt file upload reads the file into memory via `FileReader.readAsDataURL` with no size or type validation. The HTML `accept="image/*"` attribute is trivially bypassed. A multi-gigabyte file could crash the browser tab by consuming excessive memory during base64 conversion.

Fix: Add client-side validation: check `file.size` against a 5MB limit and validate `file.type.startsWith("image/")`. Also validate on the server side.

5. Medium Severity Findings

Medium severity findings include input validation gaps, logic bugs, type safety issues, and privacy concerns that should be addressed in the near term to improve overall application security and reliability.

MEDIUM

#31 Pervasive `any` Type on Supabase Client (10+ files)

Type Safety

wallet.ts, stripe.ts, vouchers.ts, battles.ts, etc. (lines varies)

Nearly every data-access file casts supabase to `any` at module scope (const sb: any = supabase). This disables ALL type checking on every query, insert, update, and delete operation. Wrong column names, wrong types, and missing tables will not be caught at compile time.

Fix: Use the generated Supabase types from @/integrations/supabase/types. Replace `const sb: any = supabase` with the properly typed client.

MEDIUM

#32 Missing Amount Validation in Financial Functions

Security

src/lib/wallet.ts, src/lib/stripe.ts (lines 102-159, 17-67)

processTransaction(), createDepositRequest(), createWithdrawalRequest(), createCheckoutSession(), and createPaymentIntent() accept amount: number with no validation for positive values, minimum amounts, or reasonable maximums. Negative or zero amounts could pass through.

Fix: Add validation: amount > 0, Number.isFinite(amount), and optional maximum limit at the start of each function.

MEDIUM

#33 Missing Input Validation in createVoucher

Security

src/lib/vouchers.ts (lines 167-185)

createVoucher() accepts type: string (not constrained), no validation on value (could be negative), and no validation on date ranges. The spread passes all fields directly to the database.

Fix: Validate type is "percentage" or "fixed", value > 0, valid_from < valid_until, and value <= 100 when type is "percentage".

MEDIUM

#34 Prediction System Lacks Duplicate/Spend Validation

Security

src/lib/esports.ts (lines 1044-1063)

makePrediction() inserts a prediction without checking if the user already predicted, if the user has enough points, or if the match is still open. Points wagered is taken directly from the client.

Fix: Use an RPC that atomically checks user points, verifies match status, prevents duplicates, and deducts wagered points.

MEDIUM

#35 Chat Message Injection

Security

`src/lib/livePlatform.ts` (lines 181-200)

`sendChatMessage()` accepts a message from the client, truncates to 500 chars but performs no sanitization. If the message is later rendered as HTML (not React JSX), it could be an XSS vector.

Fix: Ensure all rendering uses React JSX (which auto-escapes). Add server-side validation for message content and length.

MEDIUM

#36 CSV Injection in Live Report Export

Security

`src/components/livegames/LiveControlPanel.tsx` (lines 1306-1308)

Player names in the CSV export are wrapped in double quotes but not escaped. A name containing formula characters (`=CMD()`, `=HYPERLINK()`) would enable DDE/formula injection when opened in Excel.

Fix: Escape double quotes by doubling them. Consider prepending names with a tab character to prevent formula execution in spreadsheets.

MEDIUM

#37 URL.createObjectURL Memory Leak in exportToCSV

Code Quality

`src/lib/export.ts` (lines 1-28)

`exportToCSV()` creates a blob URL with `URL.createObjectURL()` but never calls `URL.revokeObjectURL()`. Each call leaks a blob URL in browser memory.

Fix: Add `URL.revokeObjectURL(url)` after `link.click()`, similar to the pattern used in `liveHistory.ts` line 149.

MEDIUM

#38 Voter Hash is Trivially Spoofable

Security

`src/lib/liveStudio.ts` (lines 112-120)

`getVoterHash()` generates a random UUID stored in `localStorage`. A user can clear `localStorage` or use multiple browser profiles to get new hashes and vote multiple times on a poll.

Fix: Ensure server-side RPC enforces per-poll uniqueness by user ID for logged-in users or IP-based rate limiting for anonymous users.

MEDIUM

#39 Prediction Resolution Pays 2x Regardless of Odds

Logic Bug

`src/lib/esports.ts` (lines 1071-1087)

`resolvePredictions()` awards `points_won`: `correct ? p.points_wagered * 2 : 0`, a flat 2x payout with no odds calculation. Every correct prediction is equally profitable regardless of how obvious the outcome was.

Fix: If 2x is intentional, document it. Otherwise, implement a proper odds system. At minimum, batch the updates for performance.

MEDIUM

#40 Accept Invitation Has TOCTOU Race

Logic Bug

`src/lib/leagues.ts` (lines 515-533)

`acceptInvitation()` reads the invite, checks `uses_count < max_uses`, then updates. Between the read and write, another user could accept the same invitation.

Fix: Use a single RPC that atomically checks and increments. Use a unique partial index or `ON CONFLICT` to prevent over-redemption.

MEDIUM

#41 Streak Calculation Has Timezone Bug

Logic Bug

`src/lib/gamification.ts` (lines 183-228)

`updateStreak()` uses `new Date().toISOString().split("T")[0]` for "today" (UTC) but `Date.now()` - 86400000 for "yesterday". Users in negative UTC offsets near midnight may have streak calculations fail.

Fix: Use consistent UTC throughout, or use a library like `date-fns-tz` for proper timezone-aware date comparisons.

MEDIUM

#42 Context Default Value Anti-Pattern

Code Quality

`src/contexts/AuthContext.tsx`, `ThemeContext.tsx` (lines 26, 10)

`AuthContext` is created with `{}` as `AuthContextType` and `ThemeContext` with a no-op `toggleTheme`. When used outside their providers, hooks silently return fake defaults instead of throwing, masking bugs.

Fix: Use `createContext<AuthContextType | null>(null)` and throw a descriptive error in the hook when context is null.

MEDIUM

#43 LanguageContext Creates New t() Function Every Render

Performance

`src/contexts/LanguageContext.tsx` (lines 5088-5100)

The `t` function is recreated on every render of `LanguageProvider`. Since it is passed as part of the context value, every consumer re-renders even if the language has not changed.

Fix: Wrap `t` in `useCallback` with `[lang]` as dependency, or use `useMemo` for the entire context value object.

MEDIUM

#44 useGameSessionTracker Loses Data on Unmount

Logic Bug

`src/hooks/useGameSessionTracker.ts` (lines 50-67)

The tracker batches events and flushes every 2 seconds. If the component unmounts before the timer fires, pending events in `pendingRef.current` are silently lost.

Fix: Add a `useEffect` with an empty dependency array that calls `flush()` on cleanup.

MEDIUM

#45 Failed Batch Re-queued Infinitely with No Retry Limit

Logic Bug

[src/hooks/useGameSessionTracker.ts \(lines 44-47\)](#)

When flush fails, the entire batch is pushed back to pendingRef.current. On the next track() call, a new timer retries the same batch. If the error is permanent, the batch grows infinitely with each new event.

Fix: Add a retry counter or max retry limit. After N failed attempts, discard the batch and log an error.

MEDIUM

#46 IP Geolocation to Untrusted Third-Party Services

Security / Privacy

[src/hooks/useRegionalConfig.tsx \(lines 94-114\)](#)

detectUserRegion makes requests to ipwho.is and freeipapi.com, free untrusted geolocation services that can log user IPs, inject malicious responses, or be compromised. The response country_code is not validated against supported regions.

Fix: Validate the returned country code against a whitelist. Consider using a self-hosted geolocation service.

MEDIUM

#47 Missing Cancellation in loadConfig Race Condition

Logic Bug

[src/hooks/useRegionalConfig.tsx \(lines 254-279\)](#)

loadConfig is an async function called in useEffect and exposed as refreshConfig. There is no abort controller. If refreshConfig is called while loadConfig is in flight, both race and the slower one may overwrite with stale data.

Fix: Use an AbortController or mounted-ref pattern to skip state updates if the component has moved on.

MEDIUM

#48 Toast Timeout Memory Leak

Performance

[src/hooks/use-toast.ts \(lines 53-69\)](#)

The module-level toastTimeouts Map stores a setTimeout for every dismissed toast for TOAST_REMOVE_DELAY = 1000000ms (16.7 minutes). Many toasts accumulate entries for 16+ minutes each.

Fix: Reduce TOAST_REMOVE_DELAY to 5000ms. Clear the timeout when the toast is already removed from state.

MEDIUM

#49 useToast Effect Re-registers Listener on Every State Change

Performance

[src/hooks/use-toast.ts \(lines 169-177\)](#)

The useEffect has [state] in its dependency array, causing the listener to be removed and re-added on every toast state change. This causes unnecessary churn and could miss state updates.

Fix: Change dependency to [] (empty array). The useState reference is stable since it comes from useState.

MEDIUM

#50 CurrencyContext Fallback Uses MZN But Detect Initial Never Returns It

Logic Bug

[src/contexts/CurrencyContext.tsx \(lines 44-46\)](#)

When useCurrency() is called outside the provider, the fallback returns "MZN" but detectInitial() only returns "USD", "CAD", or "INR". A component running outside the provider would show prices in MZN while the rest uses a different currency.

Fix: Use "USD" as the fallback to match the default from detectInitial().

MEDIUM

#51 Unsafe Type Cast in PaymentGatewaySelector

Type Safety

[src/components/payments/PaymentGatewaySelector.tsx \(lines 79\)](#)

currency as "USD" | "CAD" | "INR" is an unsafe assertion with no runtime check. If currency is "MZN" or "BRL", it silently passes through to PayPal SDK which may malfunction.

Fix: Add a runtime check and conditionally render PayPal only for supported currencies.

6. Low Severity Findings

Low severity findings include minor code quality issues, cosmetic logic bugs, and error handling improvements. These should be addressed as part of ongoing maintenance to improve code health and developer experience.

LOW

#52 Pervasive console.error-Only Error Handling

Error Handling

wallet.ts, vouchers.ts, battles.ts, audit.ts, etc. (lines varies)

Most errors across 10+ files are logged to console.error and then silently swallowed (returning null, [], or nothing). Errors are invisible to users and there is no error reporting infrastructure.

Fix: Implement an error reporting service (Sentry, LogRocket). For user-facing operations, propagate errors to the UI with toast notifications.

LOW

#53 share.ts Swallows Native Share Errors

Error Handling

src/lib/share.ts (lines 19)

The catch {} block after navigator.share() silently swallows all errors, including genuine failures that are not AbortError (user cancelled).

Fix: Check for AbortError and re-throw other errors, or at minimum log them.

LOW

#54 oneClick.ts Stores PayPal Email in localStorage

Security

src/lib/oneClick.ts (lines 13-26)

The user PayPal email is stored in localStorage as plain text. Any XSS on the page can read this. The data is parsed without schema validation.

Fix: Add schema validation on read. Consider encrypting the stored value if sensitive.

LOW

#55 pushNotifications.ts Stores Raw User-Agent

Privacy

src/lib/pushNotifications.ts (lines 140-142)

storeSubscription() stores the full navigator.userAgent in the database, which is a privacy concern (fingerprinting) and can be 500+ chars.

Fix: Store only a sanitized, truncated version (browser name + OS) or use navigator.userAgentData API.

LOW

#56 Unused Import in vouchers.ts

Dead Code

src/lib/vouchers.ts (lines 2)

import type { TablesInsert as SBTablesInsert } is imported but never used.

Fix: Remove the unused import.

LOW

#57 authErrors.ts Email Exists Action is Wrong

Logic Bug

`src/lib/authErrors.ts (lines 89)`

When sign-up fails because the email already exists, the suggested action is "signup". It should be "retry" with a message pointing to sign-in.

Fix: Change action: "signup" to action: "retry" or remove it since the correct next step is to sign in.

LOW

#58 shuffleAnswers Can Produce Undefined Slots

Logic Bug

`src/lib/openTriviaDB.ts (lines 50-69)`

For boolean-type trivia questions, `incorrect_answers` has only 1 item, producing a 2-element array. But the return type always has `option_a` through `option_d`, so `option_c` and `option_d` will be undefined.

Fix: For boolean questions, set unused options to empty strings or handle the boolean case separately.

LOW

#59 liveBus.ts Subscriber Cleanup Does Not Remove Realtime Channel

Resource Leak

`src/lib/liveBus.ts (lines 123-149)`

The `subscribe()` cleanup removes handlers and listeners but never unsubscribes the Supabase Realtime channel. If all subscribers disconnect, the channel remains open.

Fix: Track subscriber count and call `supabase.removeChannel(rtChannel)` when it drops to zero.

LOW

#60 sort_by in getTeams Is Unchecked

Security

`src/lib/esports.ts (lines 474-486)`

`getTeams()` passes `filters.sort_by` directly to `.order()` without validating against a whitelist of allowed column names.

Fix: Whitelist allowed sort columns. Default to a safe column if the input is not in the whitelist.

LOW

#61 distributePrizes Only Distributes to Top 3

Logic Bug

`src/lib/esports.ts (lines 1230-1259)`

`distributePrizes()` hardcodes `top3 = standings.slice(0, 3)` but `prize_distribution` could define prizes for more than 3 places. Any entries beyond key "3" are silently ignored.

Fix: Iterate over the distribution keys instead of hardcoding 3.

LOW

#62 Memory Leak: New AudioContext on Every Sound

Code Quality

`src/lib/sounds.ts (lines 2)`

audioCtx() creates a new AudioContext() on every sound effect call. Browsers limit AudioContexts (typically 6). After several sounds, the browser throws "Maximum number of AudioContexts reached" errors.

Fix: Cache the AudioContext as a module-level singleton.

LOW

#63 liveStudio.ts Inconsistent Error Handling

Error Handling

src/lib/liveStudio.ts (lines 83-88, 101-106)

removeLiveLink() and deletePoll() do not check for errors at all. If the delete fails, the UI will not know.

Fix: Return the error so the caller can handle it, or at minimum log it.

LOW

#64 LanguageContext Is a 5118-Line File

Code Quality

src/contexts/LanguageContext.tsx (lines all)

The entire translation dictionary is defined inline in the context file, making it extremely large, hard to maintain, and causing all translations to be bundled even if only one language is needed.

Fix: Extract translations into separate locale files and import them dynamically.

LOW

#65 Profile Edit Has No Input Validation

Security

src/pages/Profile.tsx (lines 102-116)

handleSave sends displayName and phone to Supabase without any client-side validation: no length limits, no format checks, no sanitization.

Fix: Add validation: trim, max length (100 chars for name, 20 for phone), and basic phone format check.

LOW

#66 Register.tsx Email Validation Is Weak

Code Quality

src/pages/Register.tsx (lines 105)

email.includes("@") is the only email validation. Strings like "@", "@@", or "foo@" would pass. The HTML type="email" can be bypassed.

Fix: Use a proper email regex or validation library (e.g., z.string().email() from Zod).

LOW

#67 SafeGameLoader Stale Closure

Logic Bug

src/components/livegames/SafeGameLoader.tsx (lines 47-61)

The _load method has a condition `!this.\_factory` that is always false at that point (just assigned), making the stale load detection dead code.

Fix: Fix the condition to `if (this.\_factory !== this.props.factory) return;` (remove the `&& !this.\_factory` part).

LOW

#68 Navbar Realtime Subscription Does Not Handle User Changes

Logic Bug

`src/components/Navbar.tsx (lines 124-130)`

The `live_sessions` realtime subscription has no dependency on `user` in its `useEffect`. It subscribes once on mount and never resubscribes if the user changes or the channel disconnects.

Fix: Add explicit error handling and reconnection logic for realtime channels.

LOW

#69 Hardcoded Payment Credentials in DepositModal

Code Quality

`src/components/wallet/DepositModal.tsx (lines 23-33)`

Phone numbers, IBANs, PIX keys, PayPal addresses, and Bitcoin wallet addresses are hardcoded in the component. Changing them requires a code deploy.

Fix: Move payment destination details to a configuration source (database table or environment variables).

LOW

#70 Markdown Export with Unsanitized Player Names

Security

`src/components/livegames/LiveControlPanel.tsx (lines 1417)`

In `copyAsMarkdown`, player names are directly interpolated into markdown table rows. Markdown allows HTML injection which could lead to XSS in markdown renderers that do not sanitize.

Fix: Escape HTML special characters from names before interpolation.

7. Issue Distribution Summary

Issues by Category

Category	CRITICAL	HIGH	MEDIUM	LOW	Total
Security	9	10	6	4	29
Logic Bug	2	5	6	5	18
Type Safety	0	1	3	0	4
Error Handling	0	1	0	5	6
Code Quality	0	0	2	4	6
Performance	0	0	2	0	2
Privacy	0	0	1	1	2
Dead Code	0	0	0	1	1
Resource Leak	0	0	0	1	1
Data Loss	0	0	1	0	1

Issues by Module

Module	Issues	Critical	Key Concern
src/lib/	35	4	Race conditions, SQL injection, XSS, broken features
src/contexts/	15	3	CSS injection, role escalation, silent errors
src/hooks/	12	2	Missing auth, infinite retry, stale closures
src/components/	14	3	XSS, SVG upload, file upload, payment issues
src/pages/	8	3	Open redirect, weak passwords, no rate limiting
src/layouts/	2	1	No server-side auth verification

8. Priority Remediation Roadmap

Phase 1: Immediate (Week 1-2)

These issues represent active exploitable vulnerabilities that must be fixed before any production deployment involving real financial transactions. The SQL injection in esports placements (issue #1) can be exploited by any authenticated user to execute arbitrary database commands. The three race conditions (issues #2-4) in voucher redemption, wallet withdrawal, and battle wager creation could allow users to double-spend funds or redeem limited-use vouchers multiple times. The CSS injection vulnerabilities (issues #5-6) enable stored XSS from any user with database write access to the regions or regional_branding tables. The role escalation (issue #7) and open redirect (issues #8-9) could be chained for a full account takeover attack: an attacker registers with admin metadata, gets redirected to a phishing page that captures OAuth tokens, and gains persistent admin access.

For each critical issue, the recommended approach is: (a) add server-side validation and atomic operations via PostgreSQL functions or Supabase Edge Functions, (b) remove client-side trust for security-sensitive operations, (c) add input sanitization for all user-controlled data before DOM injection, and (d) implement proper RLS policies on all authorization-sensitive tables.

Phase 2: Short-Term (Week 3-4)

Address all HIGH severity issues. Fix the SVG and arbitrary file upload vulnerabilities by adding server-side content validation and removing SVG from the allowed upload types. Implement proper authorization checks in updateBattleStatus and translation mutations. Add CAPTCHA and rate limiting to all authentication pages. Fix the withdrawal modal to prevent double-submission. Add server-side amount verification for Stripe payment recording. Fix the clip likes bug and implement proper duel vote deduplication. Replace empty catch blocks with proper error handling in the authentication flow.

Phase 3: Medium-Term (Month 2)

Systematically eliminate the pervasive ``const sb: any = supabase`` pattern across all 10+ data-access files. This is a significant effort but is essential for catching schema mismatches and typos at compile time. Add input validation to all financial functions (amount > 0, finite checks). Fix the game session tracker to flush on unmount and implement retry limits. Add abort controllers to all async hooks to prevent race conditions. Implement proper error reporting infrastructure (Sentry or equivalent) to replace the current console.error-only pattern.

Phase 4: Ongoing Maintenance

Address LOW severity findings as part of regular development cycles. Extract the 5000+ line `LanguageContext` into separate locale files. Cache the `AudioContext` singleton to prevent browser limit errors. Add proper email validation. Implement input validation on all form submissions. Clean up dead code and unused imports. Add reconnection logic for realtime subscriptions. Move hardcoded payment credentials to configuration sources.